

C++ FOR EMBEDDED SYSTEMS & ROBOTICS

Build Real Systems. Solve Real Problems. Think Like an Engineer.

A practical beginner-to-builder course that teaches C++ programming through embedded systems, electronics, automation, and robotics — from your first program to building real working projects.

24

WEEKS

12

LEVELS

12+

PROJECTS

0

EXPERIENCE NEEDED

SENSE



THINK



ACT

COURSE STRUCTURE: 24 WEEKS • 12 LEVELS • 12+ HANDS-ON PROJECTS

Absolute beginners welcome — ages 12 and up

By Abdullahi Oladipupo Musliudeen

Algorise Tech Explorers

www.muwatta.com.ng

Quick Start

QUICK START

Getting Started Today

You do not need any experience. You will learn by building real things: a blinking LED, a moving robot, an irrigation system, a security alarm. Every project solves a real problem.

What You Need

- ▶ Computer with VS Code installed
- ▶ Arduino Uno or compatible
- ▶ USB cable to connect Arduino to computer
- ▶ Breadboard, wires, LEDs, resistors

PART 1: Setting Up VS Code for Arduino & C++ (Week 0 Prep)

VS Code is a free, professional code editor. This section walks you through installing it, adding C++ support, and connecting your Arduino board.

Step 1: Download & Install VS Code

Visit code.visualstudio.com. Download the version for your operating system (Windows, macOS, Linux). Run the installer and follow the prompts. Launch VS Code when installation completes.

Step 2: Install Essential Extensions

Extensions add features to VS Code. You need three extensions for Arduino and C++ development:

- ▶ Arduino Extension (by Microsoft) — enables VS Code to communicate with Arduino boards
- ▶ C/C++ Extension (by Microsoft) — enables code completion, debugging, and syntax highlighting
- ▶ Code Runner (by Jun Han) — run code snippets quickly (optional but useful)

How to install an extension: Click the Extensions icon on the left sidebar (looks like four squares). Search for the extension name. Click Install.

Step 3: Install Arduino IDE (The Backend)

VS Code needs the Arduino IDE installed on your computer to compile and upload code to the board. This is separate from VS Code itself.

- ▶ Visit arduino.cc/software
- ▶ Download Arduino IDE 2.0 or later
- ▶ Run the installer and complete the setup

Quick Start

- ▶ Launch Arduino IDE once to finish setup, then you can close it (VS Code will use it in the background)

Step 4: Test the Setup

Open VS Code. Create a new file (File → New File). Write this code:

```
1 void setup() {  
2   Serial.begin(9600);  
3   Serial.println("Hello, Engineer!");  
4 }  
5  
6 void loop() {  
7   // Code here runs forever  
8 }
```

Connect your Arduino with a USB cable. Save the file as test.ino. Click the Arduino icon in VS Code's sidebar. Select your board and port. Click Upload. You should see "Uploading..." then "Upload successful".

Step 5: Using the Serial Monitor (For Debugging)

The Serial Monitor shows messages your code sends. In VS Code: click the Arduino icon, then click "Open Serial Monitor". Select the correct port. You will see any Serial.println() output from your code in real time.

Contents

CONTENTS

Table of Contents

PART 1 VS Code Setup

PART 2 Course Overview & Philosophy

PHASE 1 Programming Foundations (Weeks 1-4)

- Level 1 — Thinking Like an Engineer
- Level 2 — Variables and Data Types
- Level 3 — Making Decisions (If-Else)
- Level 4 — Loops and Repetition

PHASE 2 Functions, Arrays & Arduino (Weeks 5-12)

- Level 5 — Functions (Reusable Code)
- Level 6 — Arrays (Multiple Values)
- Level 7 — Arduino Fundamentals
- Level 8 — Digital Inputs
- Level 9 — Analog Input, Potentiometer & PWM
- Level 10 — Outputs: LED, Buzzer & Servo
- Level 11 — Generic Sensors
- Level 12 — Ultrasonic Sensor & Sensor Integration

PHASE 3 Automation (Weeks 13-16)

- Project 1 — Automatic Farm Irrigation
- Project 2 — Automatic Street Light
- Project 3 — Automatic Animal Feeder
- Project 4 — Security Alarm

PHASE 4 Robotics (Weeks 17-20)

- Motor Control Fundamentals
- Project 5 — Remote-Controlled Car
- Project 6 — Line-Following Robot
- Project 7 — Autonomous Obstacle-Avoiding Robot

APPENDIX Engineering Project Method, Debugging & Final Milestone

Note: this book follows the original 12-level, 24-week curriculum. Phases 5-7 (advanced robotics competition, ESP32/IoT, and the capstone project) continue the same SENSE → THINK → ACT pattern established here and are issued as a companion volume.

Overview & Philosophy

PART 2

Course Overview & Philosophy

Who This Course Is For

This course is for absolute beginners aged 12 and up. You do not need prior coding experience. You do not need to understand electricity. If you can follow instructions and ask questions, you can complete this course. That is all that is required.

Our Core Philosophy: Builders, Not Programmers

This course teaches you to build systems that solve real problems, not to memorise syntax. Every lesson starts with a real-world problem. You design a solution, write code to implement it, and test it on actual hardware. By the end, you will think like an engineer: seeing problems, designing solutions, and building things that work.

The Central Pattern: SENSE → THINK → ACT

Every project in this course follows the same three-step pattern:

- ▶ SENSE — a sensor reads the environment (temperature, distance, light, moisture)
- ▶ THINK — your C++ code processes that reading and makes a decision (if-else, loops)
- ▶ ACT — an actuator (motor, pump, light, buzzer) does something based on that decision

Example — an automatic street light: SENSE: an LDR measures brightness. THINK: if brightness is below a threshold, the light should turn on. ACT: an LED (or a relay controlling a lamp) turns on. This same three-step pattern applies whether you are building an irrigation system, a security alarm, or a robot.

Programming Foundations

PHASE 1 · WEEKS 1-4

Programming Foundations

Before you build anything physical, you need to learn how to think like an engineer. This phase teaches you the patterns all engineers use to solve problems: breaking them into steps, making decisions, and repeating actions. By the end, you will write C++ code that makes decisions and repeats logic. No hardware yet — just you, your computer, and the fundamental thinking patterns.

LEVEL 1 · WEEK 1

How Engineers Think

Algorithms, Flowcharts & Your First C++ Program

LEARNING OBJECTIVES

Explain what a problem, algorithm, and program are; understand the Input → Process → Output model; draw a simple flowchart; explain why computers require precise instructions; write and run a basic C++ program; explain the purpose of `#include <iostream>`, `cout`, `endl`, `main()`, and `return 0`; and make simple changes to a C++ program.

1. The Big Idea: How Engineers Think

Before learning how to write C++ code, we first need to understand how programmers and engineers think about problems.

Imagine that a farmer wants a gate that automatically opens during the day and closes at night.

A programmer should not immediately start writing code. The first step is to understand the problem and determine what information the system needs, what decision it should make, and what action should follow.

- ▶ The system may need to know whether it is day or night. This is the input.
- ▶ The computer then decides: if it is day, the gate should open; if it is night, the gate should close. This is the process.
- ▶ Finally, the gate motor performs the required action. This is the output.

Therefore, we can represent the basic structure as:

INPUT → PROCESS → OUTPUT

This pattern is found in almost every computer system and automated machine. A traffic light, washing machine, automatic door, irrigation system, and robot all receive information,

Programming Foundations

process that information, and produce an action or result.

The important lesson is that good programming begins with good thinking.

2. What Is an Algorithm?

An algorithm is a step-by-step method for solving a problem or completing a task.

For the farmer's automatic gate, an algorithm could be:

- ▶ Start.
- ▶ Check the light sensor.
- ▶ Determine whether it is dark.
- ▶ If it is dark, close the gate.
- ▶ If it is not dark, open the gate.
- ▶ Repeat the process.

Notice that we have not written any C++ yet. We have simply described what the system should do.

This is important because programming is not simply about typing code. Code is the implementation of a solution that has already been thought through. You can therefore write an algorithm without knowing any programming language.

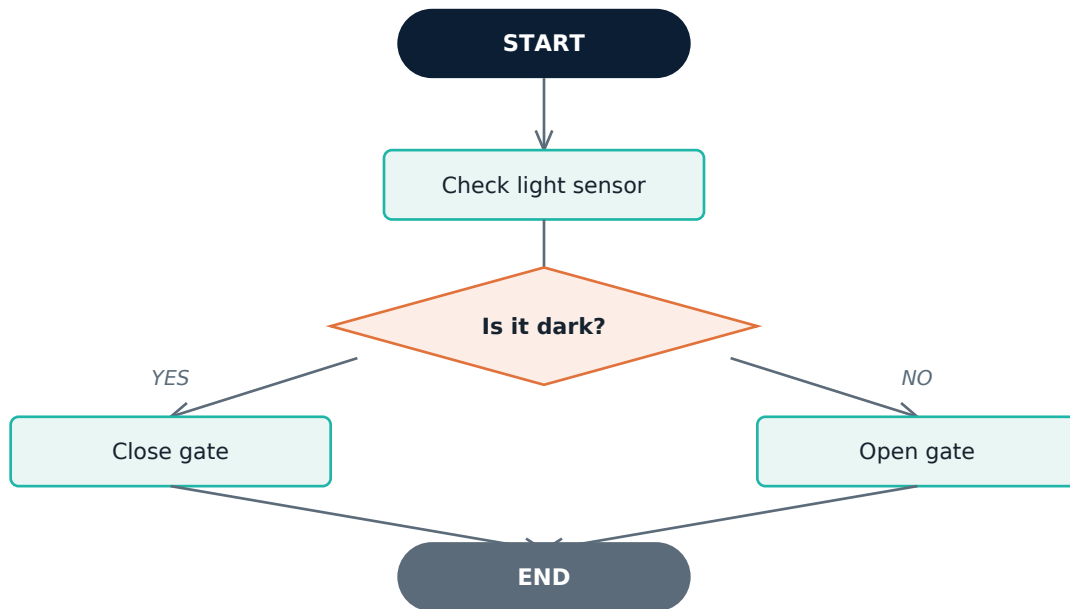
3. What Is a Flowchart?

A flowchart is a visual representation of an algorithm. It allows us to see the steps and decisions in a process before writing the program.

The four basic flowchart symbols we will use are the oval (Start or End), the rectangle (an action or process), the diamond (a decision), and the arrow (the direction of the process).

For the automatic gate, the flow can be represented as:

Programming Foundations



The diamond is particularly important because it represents a decision. Computers constantly make decisions based on conditions — for example: is the temperature too high? Is the soil dry? Is someone approaching the door? Is the tank empty? Is the light level low? These decisions will become important when we start controlling real sensors and machines.

Programming Foundations

4. From Algorithm to Program

Once we understand the problem and have designed our algorithm, we can turn our instructions into a program. A program is a set of instructions written in a programming language that a computer can execute. In this course, we are learning C++.

C++ is widely used in areas such as robotics, embedded systems, games, high-performance software, and systems programming.

We are not trying to learn all of C++ in the first lesson. Our goal is to understand how to communicate simple instructions to a computer and gradually use those skills to control real systems.

5. Your First C++ Program

Here is a simple C++ program:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Hello, I am a builder!" << endl;
6      return 0;
7  }
```

Do not try to memorise the whole program immediately. Instead, let us understand what each part means.

6. #include <iostream>

The first line gives our program access to basic input and output tools. `iostream` stands for input/output stream. We need it because we want our program to communicate with us through the terminal — we will eventually use `cout` to display information and `cin` to receive information from the keyboard.

Think about building with LEGO. Before you can use a particular LEGO piece, that piece needs to be available. In a similar way, `#include <iostream>` makes the basic input/output tools available to our program.

7. using namespace std

C++ has a standard library containing many useful tools. These tools are organised under a namespace called `std`. Normally, we could write `std::cout`. Instead, because we have written `using namespace std;`, we can simply write `cout`. We will study namespaces more deeply later.

8. main()

Programming Foundations

Every normal C++ program needs a starting point. The `main()` function is where the program begins executing — think of it as saying “start running my program here.” The instructions placed between the braces `{ }` belong to `main()`. We will study functions properly later; for now, remember that `main()` is the starting point of a C++ program.

9. `cout`

`cout` is used to display output. Whenever we want our program to communicate information to the user through the terminal, `cout` is one of the basic tools we use.

```
1 cout << "Welcome to C++";  
2 cout << "I am learning programming";
```

10. What Does `<<` Mean?

The symbol `<<` is called the stream insertion operator. At this level, you can think of it simply as “display this.” Later, we will understand the technical meaning of the operator more deeply.

11. Why Are There Quotation Marks?

Quotation marks tell C++ that what is inside them should be treated as text.

```
1 cout << "My name is Ali";
```

Output

```
My name is Ali
```

Numbers can also be displayed without quotation marks:

```
1 cout << 25;
```

Output

```
25
```

This difference will become important when we begin working with variables, calculations, and different data types.

12. `endl`

`endl` moves the output to the next line.

Programming Foundations

```
1 cout << "Hello" << endl;  
2 cout << "Welcome";
```

```
Hello  
Welcome
```

13. return 0

At the end of our program we have `return 0;` — this tells the operating system that the `main()` function has finished successfully. We will understand `return` more deeply when we learn about functions.

Programming Foundations

14. Putting Everything Together

Now look at the complete program again:

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5  cout << "Hello, I am a builder!" << endl;
6  return 0;
7  }
```

Part	What it does
#include	Provides input/output tools
using namespace std;	Allows convenient use of standard library names
int main()	Starting point of the program
cout	Displays information
endl	Moves to a new line
return 0	Program finished successfully

You do not need to memorise every technical detail at this stage. The important thing is to understand what each part is doing.

15. A Small Comparison With Python

You may eventually notice that different programming languages can perform the same task using different syntax. Python uses `print("Hello")` while C++ uses `cout << "Hello";` — both instructions essentially mean “display ‘Hello’.” This teaches us an important programming lesson: programming concepts are bigger than programming languages. The idea of output exists in both Python and C++; the syntax is simply different. We are learning C++ now, so our focus should remain on understanding C++ syntax and programming concepts — the Python example is only a comparison.

16. Your First Experiment

Now it is time to modify the program yourself. Change:

```

1  cout << "Hello, I am a builder!" << endl;
```

to something like:

Programming Foundations

```
1 cout << "My name is _____.\" << endl;  
2 cout << "I want to become a _____.\" << endl;
```

Run the program and observe the result. After that, change the messages again. The goal is not simply to copy the teacher's code — change it, run it, observe what happens. This is how programmers begin to develop confidence with code.

17. How Do We Run C++?

A C++ program is normally saved with the `.cpp` file extension, for example `hello.cpp`. However, the computer does not normally execute the source code directly — a compiler translates the C++ source code into an executable program that the computer can run. For example, with the `g++` compiler:

```
1 g++ hello.cpp -o hello
```

On Linux or macOS, the program can then be run with `./hello`. On Windows, the executable may be `hello.exe`. The basic process is:

C++ Source Code → Compiler → Executable Program → Run → Output

You do not need to understand compilers deeply yet. Just remember: we write C++, the compiler translates it, and the computer runs the result.

Programming Foundations

18. Classroom Activities

Activity 1 — Algorithm. Write an algorithm for making tea. As you write the steps, identify the Input, Process, and Output. Your algorithm should contain at least one decision — for example, you might need to decide whether the water has boiled before continuing.

Activity 2 — Flowchart. Convert your tea-making algorithm into a flowchart. Use the correct shapes for Start/End, Process, Decision, and Direction.

Activity 3 — Real-World Systems. Choose one of the following systems: traffic light, washing machine, automatic door, or vending machine. Explain its Input → Process → Output — what information the system receives, what decision or processing takes place, and what action or result follows.

Activity 4 — C++ Output. Write a C++ program that displays your name, school, favourite subject, and something you would like to build. Use `cout` and `endl`.

Activity 5 — Explain in Your Own Words. Without looking at your notes, explain what each of the following does: `#include <iostream>`, `cout`, `endl`, `main()`, `return 0`. Do not simply recite the definitions — explain them as if you were teaching another student.

ENGINEER'S CHALLENGE

Return to our farmer's automatic gate. Before writing any code, think like an engineer. Ask yourself: What information does the system need? This is the INPUT. What decision should the computer make? This is the PROCESS. What should happen after the decision? This is the OUTPUT. Now write the algorithm for the automatic gate — do not worry about writing C++ yet. First make sure the logic is correct. The habit we want to build throughout this course is simple: think first, code second.

Programming Foundations

LEVEL 2 · WEEK 2

Variables and Data Types

Teaching the Computer to Store Information

LEARNING OBJECTIVES

Understand what variables and data types are; create simple variables in C++; use `cin` to receive information; and choose suitable data types for simple problems.

THE REAL PROBLEM

Imagine we are building a small weather station. It needs to remember a temperature of 28.5°C, humidity of 65%, 12 rainy days, the location “Lagos”, and whether it is currently raining. The computer needs somewhere to keep these values — this is where variables are used.

What Is a Variable?

A variable is a named place for storing information. Think of it as a labelled container:

```
1 temperature → 28.5
2 rainyDays → 12
3 location → "Lagos"
4 isRaining → true
```

The name allows the program to find and use the value later. In C++:

```
1 int rainyDays = 12;
2 float temperature = 28.5;
```

Here, `rainyDays` and `temperature` are variables. A variable can also change:

```
1 int speed = 100;
2 speed = 150;
```

The variable is still called `speed`, but its value has changed from 100 to 150.

What Is a Data Type?

Not all information is the same. A person’s age, temperature, name, and whether a machine is running are different kinds of information. A data type tells C++ what kind of value a variable stores.

Programming Foundations

Type	Used for	Example
int	Whole numbers	int age = 16;
float	Decimal numbers	float temperature = 28.5;
bool	True or false	bool isRaining = true;
char	One character	char grade = 'A';
string	Text	string name = "Musa";

For string, we normally include `#include <string>`.

Why Does the Type Matter?

Suppose the temperature is 28.5°C. If we write:

```
1 int temperature = 28.5;
```

we are asking an integer variable to store a decimal value. The fractional part cannot be represented as an `int`, so information can be lost. Instead:

```
1 float temperature = 28.5;
```

is more appropriate. Data types also affect how values are stored in memory. This becomes particularly important in embedded systems, where a microcontroller may have very limited memory. The exact size of a type depends on the computer or microcontroller being used, so we should not assume that an `int` is always the same size everywhere.

The important principle is: choose a type that matches the kind of information you are storing.

Programming Foundations

cout and cin

In Week 1, we used `cout` to display information. Now we introduce `cin`, which allows the user to enter information into the program.

```
1  int age;
2  cout << "Enter your age: ";
3  cin >> age;
4  cout << "You are " << age << " years old.";
```

The basic idea is:

Keyboard → cin → Variable → Program → cout → Screen

This gives our programs the ability to receive information and respond to it.

Example: Simple Calculator

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5  int firstNumber;
6  int secondNumber;
7  int result;
8
9  cout << "Enter the first number: ";
10 cin >> firstNumber;
11
12 cout << "Enter the second number: ";
13 cin >> secondNumber;
14
15 result = firstNumber + secondNumber;
16
17 cout << "Result: " << result << endl;
18
19 return 0;
20 }
```

If the user enters 10 and 5, the program calculates $10 + 5 = 15$. Notice the Input → Process → Output pattern from Week 1: Input → 10, 5. Process → $10 + 5$. Output → 15.

A Small Comparison

In Python, you might write `age = 16`. In C++, you write `int age = 16`; — C++ explicitly tells the computer that `age` is an integer. This is one reason C++ requires us to think carefully about the type of information we are storing. We are not learning Python here; this comparison simply helps you understand why C++ looks different.

Assignment

Programming Foundations

Task 1. Create variables for your name, age, height, and whether you are a student. Choose an appropriate type for each and display them using `cout`.

Task 2. A robot needs to store its motor speed, battery voltage, motor pin number, and whether it is moving. Choose an appropriate data type for each and explain why.

Task 3. Write a program that asks the user for two numbers, adds them, and displays the answer using `cin` and `cout`.

Task 4. Try:

```
1  int temperature = 28.5;  
2  cout << temperature;
```

Then change `int` to `float`. Observe the difference and explain why it happened.

ENGINEER'S CHALLENGE

A soil-monitoring system needs to store: Moisture = 32.5%, Sensor pin = 34, Dry threshold = 40%, Pump = OFF. Create variables for these values and choose an appropriate data type for each. Think about this: if moisture falls below the dry threshold, what should the system do? Next week, we will teach the computer how to make that decision using conditions and if statements.

Programming Foundations

LEVEL 3 · WEEK 3

Making Decisions

Teaching the Computer to Make Choices

LEARNING OBJECTIVES

Use `if`, `else if`, and `else` to make decisions; compare values using comparison operators; combine conditions using logical operators; and predict what simple code will do.

THE REAL PROBLEM

An automatic greenhouse door should open when the temperature is above 28°C AND humidity is above 60%. Otherwise, it should remain closed. The computer needs to ask: is the temperature above 28? AND is the humidity above 60? Only when both answers are true should the door open. This is called conditional logic.

if and else

We make everyday decisions using conditions: IF it is raining, take an umbrella. ELSE, don't take one. C++ can express the same idea:

```
1  if (raining) {
2  cout << "Take an umbrella.";
3  } else {
4  cout << "No umbrella needed.";
5  }
```

The computer checks the condition. If it is true, it executes the first block. If it is false, it executes the else block.

Example: Automatic Light

```
1  int lightLevel = 400;
2  if (lightLevel < 500) {
3  cout << "It is dark. Turn light ON." << endl;
4  } else {
5  cout << "It is bright. Keep light OFF." << endl;
6  }
```

If `lightLevel` is 400, the condition `lightLevel < 500` is true, so the light turns on.

Comparison Operators

Comparison operators allow the computer to ask questions about values.

Programming Foundations

Operator	Meaning	Example
==	Equal to	age == 16
!=	Not equal to	age != 16
<	Less than	temp < 28
>	Greater than	temp > 28
<=	Less than or equal	score <= 50
>=	Greater than or equal	score >= 50

The result of a comparison is either true or false. For example, `5 > 3` is true, and `5 == 3` is false.

Important: = vs ==

These are different: `age = 16;` means “store 16 in age,” while `age == 16` means “is age equal to 16?” This is one of the most common beginner mistakes.

Programming Foundations

else if — More Than Two Choices

Sometimes there are more than two possibilities:

```
1  if (score >= 70) {  
2    cout << "Excellent";  
3  } else if (score >= 50) {  
4    cout << "Pass";  
5  } else {  
6    cout << "Try again";  
7  }
```

The computer checks the conditions from top to bottom and executes the appropriate block.

Combining Conditions

Sometimes one condition is not enough. C++ provides three important logical operators:

Operator	Meaning
&&	AND — both conditions must be true
	OR — at least one condition must be true
!	NOT — reverses true/false

For the greenhouse:

```
1  if (temp > 28 && humidity > 60) {  
2    cout << "Door OPEN" << endl;  
3  } else {  
4    cout << "Door CLOSED" << endl;  
5  }
```

The door opens only when both conditions are true. For OR:

```
1  if (temperature > 35 || humidity > 80) {  
2    cout << "Warning!" << endl;  
3  }
```

The warning appears if either condition is true. For NOT:

```
1  if (!isSunny) {  
2    cout << "It is not sunny." << endl;  
3  }
```

Programming Foundations

Think Before You Run

Programmers should learn to predict what their code will do. Consider:

```
1  int age = 15;
2
3  if (age >= 18) {
4  cout << "Adult";
5  } else {
6  cout << "Minor";
7  }
```

Before running it, ask: is $15 \geq 18$ true or false? It is false. Therefore, the program prints:

Minor

This skill is called tracing code. It helps you understand programs and find mistakes without always running them.

Assignment

Task 1. Write a program that asks for a student's age. If the age is below 12, print "Too young for this course". Otherwise, print "Welcome!".

Task 2. Write a program that checks whether a number is positive, negative, or zero using `if`, `else if`, and `else`.

Task 3. A robot should stop when an obstacle is less than 20 cm away and the robot is currently moving. Write the appropriate condition using `&&`.

Task 4. Write the condition for a greenhouse door to open when temperature > 28 AND humidity > 60 .

Task 5 — Trace the Code. Without running this, what will it print? Explain why.

```
1  if (5 > 3 && 10 < 15) {
2  cout << "YES";
3  } else {
4  cout << "NO";
5  }
```

ENGINEER'S CHALLENGE

A smart irrigation system should turn the pump on when soil moisture is below 40% AND the water tank is not empty. Before writing the code, express the decision in plain English. Then convert it into a C++ condition.

Programming Foundations

LEVEL 4 • WEEK 4

Loops and Repetition

LEARNING OBJECTIVES

Understand how to repeat instructions using for and while loops; trace a loop; and avoid infinite loops.

THE REAL PROBLEM

Imagine a robot checking a sensor 1,000 times. Writing the same instruction 1,000 times would be inefficient. A loop allows the computer to repeat an instruction automatically.

for Loop

Use a for loop when you know how many times you want something to happen.

```
1  for (int i = 1; i <= 10; i++) {  
2  cout << i << endl;  
3  }
```

This prints numbers 1 to 10. The three parts mean:

- ▶ `int i = 1` — start at 1
- ▶ `i <= 10` — continue while i is 10 or less
- ▶ `i++` — increase i by 1 each time

For example, a robot can repeat an action:

```
1  for (int step = 0; step < 50; step++) {  
2  moveForward();  
3  }
```

while Loop

Use a while loop when you want to continue until a condition becomes false.

```
1  while (distance > 20) {  
2  moveForward();  
3  distance = readSensor();  
4  }
```

Programming Foundations

The robot keeps moving while the distance is greater than 20. When an obstacle comes within 20 cm, the loop stops.

Avoid Infinite Loops

A loop must eventually be able to stop.

```
1 while (temperature > 40) {  
2   cout << "Temperature too high!" << endl;  
3 }
```

If temperature never changes, the condition remains true and the loop continues forever. This is called an infinite loop.

Practice

- ▶ Print “I am a builder” 10 times using a for loop.
- ▶ Count down from 10 to 1, then print “Blastoff!”.
- ▶ Write a loop that makes a robot move forward and turn right four times to trace a square.
- ▶ Print “Even” or “Odd” for numbers 1-20 using a loop and if.
- ▶ Trace the code below — what will it print?

```
1 for (int i = 0; i < 3; i++) {  
2   cout << i * 2 << endl;  
3 }
```

ENGINEER'S CHALLENGE

A water tank has a sensor. Write a while loop that keeps the pump running while the tank level is below 80%, then stops the pump when it reaches 80%.

Functions, Arrays & Arduino

PHASE 2 · WEEKS 5-12

Functions, Arrays & Arduino

This phase moves students from basic C++ programming into physical computing. They learn to organize code, work with sensor data, control electronic components, and build small Arduino systems.

LEVEL 5 · WEEK 5

Functions

LEARNING OBJECTIVES

Create reusable functions; use parameters; return values; and organize a program into smaller tasks.

THE REAL PROBLEM

A robot may need to move forward many times. Instead of writing the motor commands repeatedly, create a function called `moveForward()` and reuse it.

What Is a Function?

A function is a named block of code that performs a specific task.

```
1 void moveForward() {  
2   cout << "Robot moving forward" << endl;  
3 }  
4  
5 int main() {  
6   moveForward();  
7   moveForward();  
8   return 0;  
9 }
```

Functions can receive information through parameters:

```
1 void moveForward(int speed) {  
2   cout << "Speed: " << speed << endl;  
3 }  
4 moveForward(100);  
5 moveForward(200);
```

A function can also return a result:

Functions, Arrays & Arduino

```
1  int add(int a, int b) {  
2      return a + b;  
3  }  
4  
5  int answer = add(5, 3);  
6  cout << answer << endl;
```

Parameters go into a function. A return value comes out.

Practice

- ▶ Create `printGreeting()` and call it three times.
- ▶ Create `calculateArea(length, width)`.
- ▶ Create `moveForward(speed)`, `turnLeft(speed)` and `turnRight(speed)`.
- ▶ Create a function that calculates the average of three sensor readings.
- ▶ Create `isEven(number)` that returns true or false.

Functions, Arrays & Arduino

LEVEL 6 · WEEK 6

Arrays

LEARNING OBJECTIVES

Store multiple related values; access them using indexes; and process them with loops.

THE REAL PROBLEM

A weather station records 24 temperature readings. Instead of creating 24 separate variables, we can store them in one array.

```
1  int temperatures[5] = {28, 30, 31, 29, 27};
2  for (int i = 0; i < 5; i++) {
3      cout << temperatures[i] << endl;
4  }
```

Array positions start at 0: temperatures[0] is the first value, temperatures[1] the second value, and temperatures[4] the fifth value.

Arrays become powerful when combined with loops.

```
1  int total = 0;
2  for (int i = 0; i < 5; i++) {
3      total += temperatures[i];
4  }
5  cout << "Total: " << total << endl;
```

Practice — create arrays to:

- ▶ Calculate the average of soil-moisture readings.
- ▶ Find the highest and lowest sensor values.
- ▶ Store five motor speeds.
- ▶ Count temperatures above 25°C.
- ▶ Store four distance readings: front, back, left and right.

Functions, Arrays & Arduino

LEVEL 7 · WEEK 7

Arduino Fundamentals

Arduino is a small programmable microcontroller board. It allows C++ programs to interact with the physical world. A basic system follows INPUT → PROCESS → OUTPUT. For an automated system: SENSE → THINK → ACT.

Arduino Program Structure

```
1 void setup() {  
2   pinMode(13, OUTPUT);  
3 }  
4  
5 void loop() {  
6   digitalWrite(13, HIGH);  
7   delay(1000);  
8  
9   digitalWrite(13, LOW);  
10  delay(1000);  
11 }
```

`setup()` runs once when the Arduino starts. `loop()` runs repeatedly.

Important Functions

Function	Purpose
<code>pinMode()</code>	Configure a pin
<code>digitalWrite()</code>	Set a digital output
<code>digitalRead()</code>	Read a digital input
<code>analogRead()</code>	Read an analog value
<code>analogWrite()</code>	Produce PWM output
<code>delay()</code>	Pause the program
<code>Serial.begin()</code>	Start serial communication
<code>Serial.print()</code>	Send information to Serial Monitor
<code>Serial.println()</code>	Send information and move to a new line

Digital Pins

Digital signals normally represent two states: HIGH = ON / 1, LOW = OFF / 0.

Functions, Arrays & Arduino

Analog Pins

Analog sensors can provide a range of values. On a typical Arduino Uno, `analogRead()` returns 0 to 1023. The exact electrical behavior depends on the board and reference voltage.

Practice — build a simple LED circuit and:

- ▶ Blink the LED.
- ▶ Change the timing.
- ▶ Control two LEDs.
- ▶ Print “LED ON” and “LED OFF” to Serial Monitor.
- ▶ Experiment with removing `delay()`.

Functions, Arrays & Arduino

LEVEL 8 · WEEK 8

Digital Inputs

Before using complicated sensors, students should understand simple digital inputs.

Push Button

A button can provide information such as: Pressed → HIGH/LOW. Released → HIGH/LOW.

```
1  const int BUTTON = 2;
2  const int LED = 13;
3
4  void setup() {
5    pinMode(BUTTON, INPUT);
6    pinMode(LED, OUTPUT);
7  }
8
9  void loop() {
10   int state = digitalRead(BUTTON);
11
12   if (state == HIGH) {
13     digitalWrite(LED, HIGH);
14   } else {
15     digitalWrite(LED, LOW);
16   }
17 }
```

The exact wiring determines whether pressing the button produces HIGH or LOW.

Project: Button-Controlled Light

BUTTON → Arduino → LED

Students learn the complete process: INPUT → DECISION → OUTPUT.

ENGINEER'S CHALLENGE

Modify the project so that one button controls two LEDs.

Functions, Arrays & Arduino

LEVEL 9 · WEEK 9

Analog Input, Potentiometer & PWM

Potentiometer

A potentiometer is a variable resistor that allows students to manually produce different analog values.

```
1  int value = analogRead(A0);  
2  Serial.println(value);
```

Students should turn the potentiometer and observe how the value changes.

Controlling LED Brightness

Arduino can use PWM to control apparent brightness.

```
1  int value = analogRead(A0);  
2  int brightness = map(value, 0, 1023, 0, 255);  
3  analogWrite(9, brightness);
```

Here, `analogRead()` returns 0-1023, `map()` converts the range, and `analogWrite()` takes 0-255.

Project: Adjustable Lamp

Potentiometer → Arduino → LED brightness

Turning the knob changes the brightness.

Important Concept

`analogWrite()` on an Arduino Uno does not produce a true continuously variable voltage. It uses PWM (Pulse Width Modulation) to control the average effect of the output.

Functions, Arrays & Arduino

LEVEL 10 · WEEK 10

Outputs — LED, Buzzer & Servo

Students now learn different types of actuators.

LED

```
1 digitalWrite(13, HIGH);  
2 delay(500);  
3 digitalWrite(13, LOW);
```

Buzzer

```
1 tone(8, 1000);  
2 delay(500);  
3 noTone(8);
```

Servo Motor

A servo can move to a commanded angle.

```
1 #include <Servo.h>  
2 Servo gate;  
3  
4 void setup() {  
5   gate.attach(9);  
6 }  
7  
8 void loop() {  
9   gate.write(0);  
10  delay(1000);  
11  gate.write(90);  
12  delay(1000);  
13  gate.write(180);  
14  delay(1000);  
15 }
```

`write(0)` commands approximately 0°, `write(90)` commands approximately 90°, and `write(180)` commands approximately 180°. The actual mechanical range depends on the particular servo.

Project: Automatic Gate

Sensor/Button → Arduino → Servo

When the input condition is satisfied, the servo opens the gate.

Functions, Arrays & Arduino

LEVEL 11 · WEEK 11

Generic Sensors

Students should understand that different sensors provide different types of information.

Sensor	What It Detects	Example Project
LDR	Light level	Automatic light
IR sensor	Object / line	Line follower
Ultrasonic	Distance	Obstacle detector
Soil moisture	Soil condition	Irrigation
DHT11 / DHT22	Temperature/humidity	Weather station
PIR	Human movement	Security alarm
Rain sensor	Rain/water	Rain detector
Flame sensor	Flame/light	Fire warning
Potentiometer	Variable input	Brightness control
Push button	User input	Control system

The important skill is not memorizing sensor names. Students should ask: What does the sensor measure? What type of value does it provide? How can the Arduino use that value?

LDR — Light Sensor

An LDR changes its electrical behavior depending on the amount of light.

Functions, Arrays & Arduino

```
1  const int LDR = A0;
2  const int LED = 9;
3
4  void setup() {
5    pinMode(LED, OUTPUT);
6    Serial.begin(9600);
7  }
8
9  void loop() {
10   int light = analogRead(LDR);
11
12   Serial.println(light);
13
14   if (light < 500) {
15     digitalWrite(LED, HIGH);
16   } else {
17     digitalWrite(LED, LOW);
18   }
19
20   delay(200);
21 }
```

The threshold should be determined by observing actual sensor readings.

Project: Automatic Street Light

LDR → Arduino → LED

When the environment becomes dark, the LED turns on.

Functions, Arrays & Arduino

IR Sensor

An IR obstacle sensor commonly provides a digital signal.

```
1  const int IR = 2;
2  const int LED = 13;
3
4  void setup() {
5    pinMode(IR, INPUT);
6    pinMode(LED, OUTPUT);
7  }
8
9  void loop() {
10   int detected = digitalRead(IR);
11
12   if (detected == LOW) {
13     digitalWrite(LED, HIGH);
14   } else {
15     digitalWrite(LED, LOW);
16   }
17 }
```

Some modules use the opposite logic, so students should test the sensor and observe its output.

Project: Object Detector

IR Sensor → Arduino → LED/Buzzer

Functions, Arrays & Arduino

LEVEL 12 · WEEK 12

Ultrasonic Sensor & Sensor Integration

The HC-SR04 ultrasonic sensor measures distance. It uses TRIG to send an ultrasonic pulse and ECHO to receive the returning pulse.

Basic Code

```
1  const int TRIG = 9;
2  const int ECHO = 10;
3
4  void setup() {
5    pinMode(TRIG, OUTPUT);
6    pinMode(ECHO, INPUT);
7    Serial.begin(9600);
8  }
9
10 void loop() {
11   digitalWrite(TRIG, LOW);
12   delayMicroseconds(2);
13   digitalWrite(TRIG, HIGH);
14   delayMicroseconds(10);
15   digitalWrite(TRIG, LOW);
16
17   long duration = pulseIn(ECHO, HIGH);
18   float distance = duration * 0.0343 / 2;
19
20   Serial.print("Distance: ");
21   Serial.print(distance);
22   Serial.println(" cm");
23
24   delay(500);
25 }
```

Project: Distance Alarm

```
1  if (distance < 20) {
2    digitalWrite(LED, HIGH);
3    tone(BUZZER, 1000);
4  } else {
5    digitalWrite(LED, LOW);
6    noTone(BUZZER);
7  }
```

The system now performs SENSE → THINK → ACT.

Functions, Arrays & Arduino

ENGINEER'S CHALLENGE

Use an ultrasonic sensor to control a servo. For example: object approaches → ultrasonic detects → Arduino makes a decision → servo opens a gate.

Automation

PHASE 3 · WEEKS 13-16

Automation

Students now combine programming, sensors and actuators into complete systems.

Project 1: Automatic Farm Irrigation

THE REAL PROBLEM

Water crops when the soil becomes too dry.

Soil Sensor → Arduino → Relay → Pump

```
1  int moisture = analogRead(A0);  
2  if (moisture > 600) {  
3    digitalWrite(7, HIGH);  
4  } else {  
5    digitalWrite(7, LOW);  
6  }
```

The threshold must be calibrated for the actual sensor and soil.

ENGINEER'S CHALLENGE

Add a water-level sensor so the pump cannot run when the tank is empty.

Project 2: Automatic Street Light

LDR → Arduino → LEDs

The system measures light and automatically controls the lights.

ENGINEER'S CHALLENGE

Use three LEDs to represent different street lights and control their brightness using PWM.

Automation

Project 3: Automatic Animal Feeder

RTC → Arduino → Servo → Feeder

At a scheduled time, the servo opens the feeder, waits a few seconds and closes it.

Students learn:

- ▶ Time-based automation
- ▶ Servo control
- ▶ Conditions
- ▶ Functions
- ▶ System testing

Project 4: Security Alarm

Combine a PIR or IR sensor with an LED and buzzer.

```
1  if (motionDetected) {  
2    digitalWrite(LED, HIGH);  
3    tone(BUZZER, 1000);  
4  }
```

ENGINEER'S CHALLENGE

Add a button to arm and disarm the alarm.

Robotics

PHASE 4 · WEEKS 17-20

Robotics

The final phase applies everything learned to mobile robotics. A 2WD robot contains an Arduino, two DC geared motors, a motor driver, wheels, a caster wheel, a battery, and sensors. A motor driver is required because the Arduino should not directly power the DC motors.

Motor Control

Students create reusable movement functions:

```
1  void moveForward(int speed) {  
2    // motor commands  
3  }  
4  
5  void moveBackward(int speed) {  
6    // motor commands  
7  }  
8  
9  void turnLeft(int speed) {  
10   // motor commands  
11 }  
12  
13 void turnRight(int speed) {  
14   // motor commands  
15 }  
16  
17 void stopRobot() {  
18   // stop motors  
19 }
```

These functions become the basic vocabulary of the robot.

Robotics

Project 5: Remote-Controlled Car

A Bluetooth controller sends single-character commands:

Command	Action
F	Forward
B	Backward
L	Left
R	Right
S	Stop

Arduino receives the command and calls the correct movement function. This teaches INPUT → DECISION → ACTION.

Project 6: Line-Following Robot

Use two or more IR sensors to detect a line. A simple logic could be:

- ▶ Left sensor detects line → turn left
- ▶ Right sensor detects line → turn right
- ▶ Neither detects line → move forward
- ▶ Both detect line → stop / adjust

Students learn how multiple sensors can work together to control a robot.

Robotics

Project 7: Autonomous Obstacle-Avoiding Robot

The ultrasonic sensor allows the robot to detect obstacles.

- ▶ SENSE — measure the distance ahead
- ▶ THINK — if the obstacle is too close, check possible directions
- ▶ ACT — turn toward the direction with more available space

```
1  long distance = readDistance();
2  if (distance > 20) {
3    moveForward(150);
4  } else {
5    stopRobot();
6
7    if (leftDistance > rightDistance) {
8      turnLeft(150);
9    } else {
10     turnRight(150);
11   }
12 }
```

The robot continuously repeats SENSE → THINK → ACT → SENSE → THINK → ACT. This is the foundation of autonomous robotics.

Engineering Method & Debugging

APPENDIX

Engineering Project Method, Debugging & the Final Milestone

Engineering Project Method

Every project should follow the same process:

Step	Question
1. PROBLEM	What real problem are we solving?
2. SENSE	What information does the system need?
3. THINK	What decision should the program make?
4. ACT	What should the hardware do?
5. TEST	Does the system behave as expected?
6. IMPROVE	What can be changed or added?

Students should write the algorithm in plain English before writing the code.

Systematic Debugging

When something fails, do not simply say “it doesn’t work.” Identify where the problem is:

- ▶ SENSE — is the sensor giving the correct reading?
- ▶ THINK — is the condition or decision correct?
- ▶ ACT — is the LED, servo, motor or buzzer responding correctly?

Test one part at a time. For example:

```
1 Serial.println(sensorValue);
```

can help determine whether the sensor is actually producing the expected value.

The debugging cycle is:

Observe → Identify → Hypothesize → Change ONE thing → Test → Repeat

Common problems include:

- ▶ Wrong pin numbers
- ▶ Loose wires
- ▶ Incorrect sensor thresholds
- ▶ Incorrect HIGH/LOW assumptions
- ▶ Insufficient power

Engineering Method & Debugging

- ▶ Missing common ground
- ▶ Incorrect conditions
- ▶ Motor interference or resets

Engineering Method & Debugging

Final Engineering Milestone

By the end of this curriculum, students should be able to take a real problem and build a working prototype. They should understand the complete relationship:

SENSOR → INPUT → C++ PROGRAM → DECISION → ACTUATOR

For autonomous robotics:

SENSE → THINK → ACT → REPEAT

The objective is not simply to memorize Arduino functions. Students should learn how software, electronics, sensors and actuators work together to solve real problems.